

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
df1 = pd.read_csv("/content/digit_recognizer_train.csv")
```

```
df1.shape
```

```
(42000, 785)
```

```
df1.sample(5)
```

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775
32581	8	0	0	0	0	0	0	0	0	0	...	0	0
29216	0	0	0	0	0	0	0	0	0	0	...	0	0
2312	1	0	0	0	0	0	0	0	0	0	...	0	0
3329	3	0	0	0	0	0	0	0	0	0	...	0	0
2543	3	0	0	0	0	0	0	0	0	0	...	0	0

```
5 rows × 785 columns
```

```
df1.info()
```

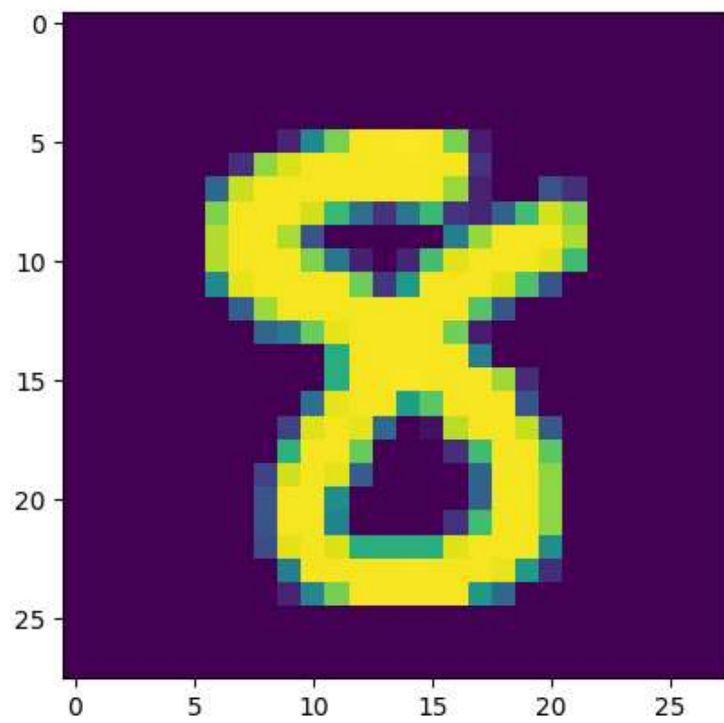
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42000 entries, 0 to 41999
Columns: 785 entries, label to pixel783
dtypes: int64(785)
memory usage: 251.5 MB
```

```
df1.isnull().sum().sum()
```

```
np.int64(0)
```

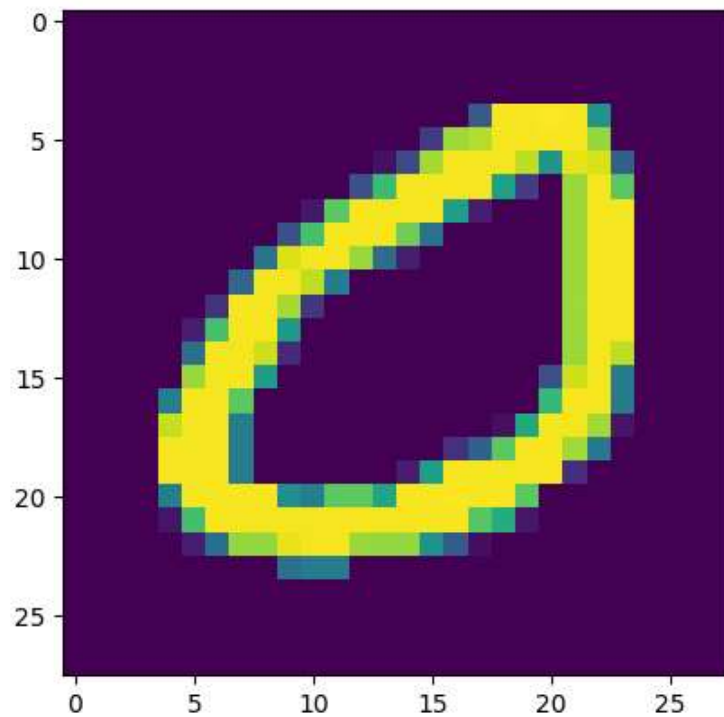
```
plt.imshow(df1.iloc[3185,1:].values.reshape(28,28))
```

```
<matplotlib.image.AxesImage at 0x7e2bbbbe7440>
```



```
plt.imshow(df1.iloc[3299,1:].values.reshape(28,28))
```

```
<matplotlib.image.AxesImage at 0x7e2bbc3a9610>
```



```
X = df1.drop('label', axis=1)
y = df1["label"]
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
X_train.shape
```

```
(33600, 784)
```

```
X_test.shape
```

```
(8400, 784)
```

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?

```
KNeighborsClassifier()
```

```
y_pred = knn.predict(X_test)
print(round(accuracy_score(y_test, y_pred),2))
print(classification_report(y_test, y_pred))
```

```
0.96
```

	precision	recall	f1-score	support
0	0.98	1.00	0.99	816
1	0.94	1.00	0.97	909
2	0.99	0.94	0.96	846
3	0.96	0.96	0.96	937
4	0.98	0.97	0.98	839
5	0.96	0.95	0.96	702
6	0.97	0.99	0.98	785
7	0.96	0.96	0.96	893
8	0.99	0.93	0.96	835
9	0.94	0.95	0.95	838
accuracy			0.96	8400
macro avg	0.97	0.96	0.96	8400
weighted avg	0.97	0.96	0.96	8400

```
for k in range(1, 11):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    print(f"K={k}, Accuracy={round(accuracy_score(y_test, knn.predict(X_test)),3)}")
```

```
K=1, Accuracy=0.969
K=2, Accuracy=0.962
K=3, Accuracy=0.967
K=4, Accuracy=0.964
K=5, Accuracy=0.965
K=6, Accuracy=0.964
K=7, Accuracy=0.963
K=8, Accuracy=0.963
K=9, Accuracy=0.961
K=10, Accuracy=0.962
```

Principle compound analysis (PCA)

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.fit_transform(X_test)
```

```
pca = PCA(n_components=None)
```

```
X_train = pca.fit_transform(X_train)
X_test = pca.fit_transform(X_test)
```

```
X_train.shape
```

```
(33600, 784)
```

```
X_test.shape
```

```
(8400, 784)
```

```
pca1 = PCA(n_components=100)
```

```
X_train_trf = pca1.fit_transform(X_train)
```

```
X_test_trf = pca1.fit_transform(X_test)
```

```
X_train_trf.shape
```

```
(33600, 100)
```

```
X_test_trf.shape
```

```
(8400, 100)
```

```
knn = KNeighborsClassifier()
```

```
knn.fit(X_train_trf, y_train)
```

▼ KNeighborsClassifier ⓘ ?

```
KNeighborsClassifier()
```

```
y_pred1 = knn.predict(X_test_trf)
```

```
print(round(accuracy_score(y_test, y_pred1),3))
```

```
print(classification_report(y_test, y_pred1))
```

```
0.642
```

	precision	recall	f1-score	support
0	0.80	0.86	0.83	816
1	0.85	0.80	0.82	909
2	0.82	0.74	0.78	846
3	0.71	0.59	0.64	937
4	0.36	0.31	0.33	839
5	0.51	0.51	0.51	702
6	0.85	0.81	0.83	785
7	0.84	0.70	0.76	893
8	0.35	0.49	0.41	835
9	0.48	0.60	0.53	838
accuracy			0.64	8400
macro avg	0.66	0.64	0.64	8400
weighted avg	0.66	0.64	0.65	8400

```
pca2 = PCA(n_components=200)
```

```
X_train_trf1 = pca2.fit_transform(X_train)
```

```
X_test_trf1 = pca2.fit_transform(X_test)
```

```
X_train_trf1.shape
```

```
(33600, 200)
```

```
knn = KNeighborsClassifier()  
knn.fit(X_train_trf1, y_train)
```

▼ KNeighborsClassifier ⓘ ?

```
KNeighborsClassifier()
```

```
y_pred2 = knn.predict(X_test_trf1)  
print(round(accuracy_score(y_test, y_pred2),3))  
print(classification_report(y_test, y_pred2))
```

```
0.644
```

	precision	recall	f1-score	support
0	0.76	0.88	0.82	816
1	0.78	0.88	0.83	909
2	0.85	0.67	0.75	846
3	0.73	0.63	0.67	937
4	0.37	0.35	0.36	839
5	0.49	0.48	0.48	702
6	0.87	0.77	0.82	785
7	0.84	0.69	0.76	893
8	0.39	0.42	0.41	835
9	0.46	0.63	0.53	838
accuracy			0.64	8400
macro avg	0.65	0.64	0.64	8400
weighted avg	0.66	0.64	0.65	8400

```
# transform to 2d coordinate system
```

```
pca1 = PCA(n_components=2)  
X_train_trf = pca1.fit_transform(X_train)  
X_test_trf = pca1.fit_transform(X_test)
```

```
X_train_trf
```

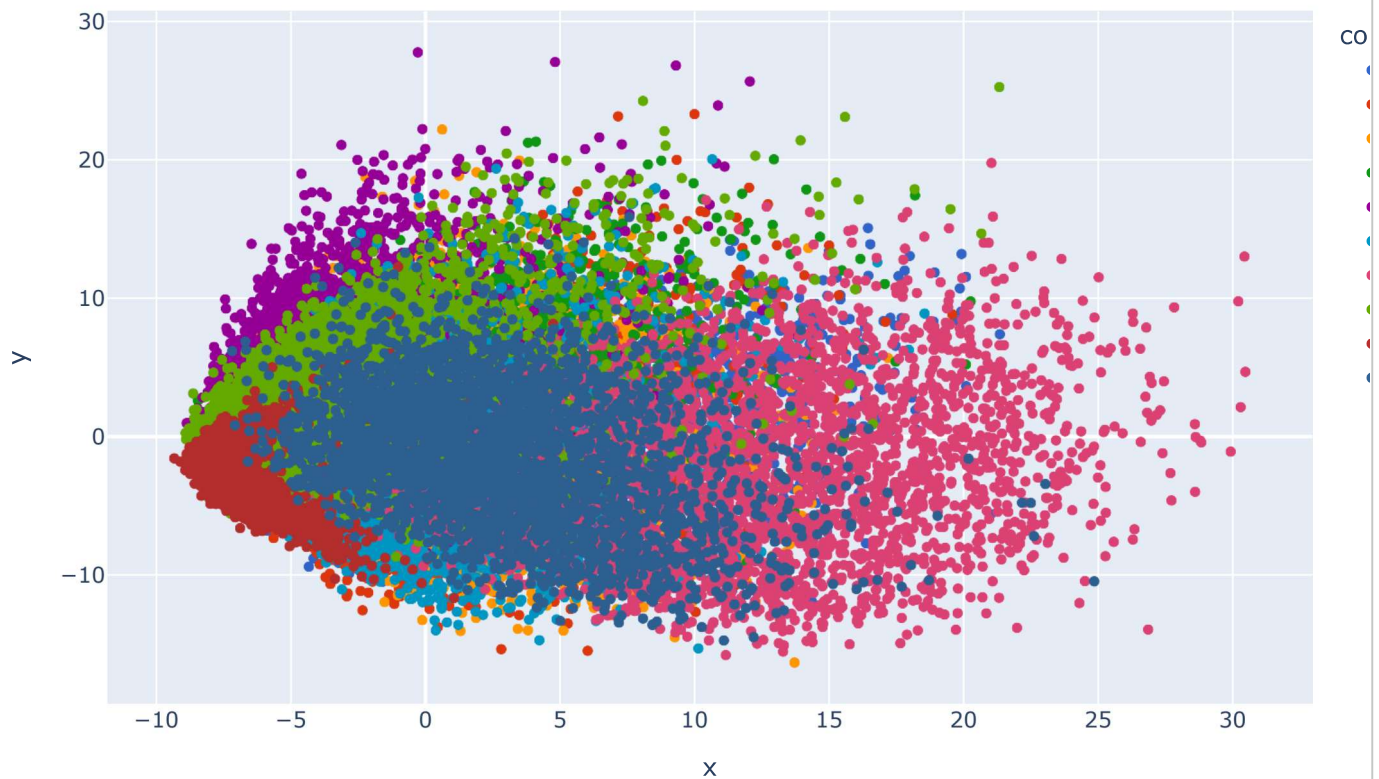
```
array([[ -2.71863253, -0.48982409],  
       [ -0.67698015, -6.75362491],  
       [ -3.03323281,  6.50981624],  
       ...,  
       [  2.14883643,  0.78079444],  
       [  1.05957006,  0.94767312],  
       [17.70258446,  1.96188185]])
```

```
X_train_trf.shape
```

```
(33600, 2)
```

```
import plotly.express as px
```

```
y_train_trf = y_train.astype(str)  
fig = px.scatter(x=X_train_trf[:,0],y= X_train_trf[:,1],color = y_train_trf,color_discrete_sequence=r  
fig.show()
```



```
# tranforming in 3D
```

```
pca1 = PCA(n_components=3)
X_train_trf = pca1.fit_transform(X_train)
X_test_trf = pca1.fit_transform(X_test)
```

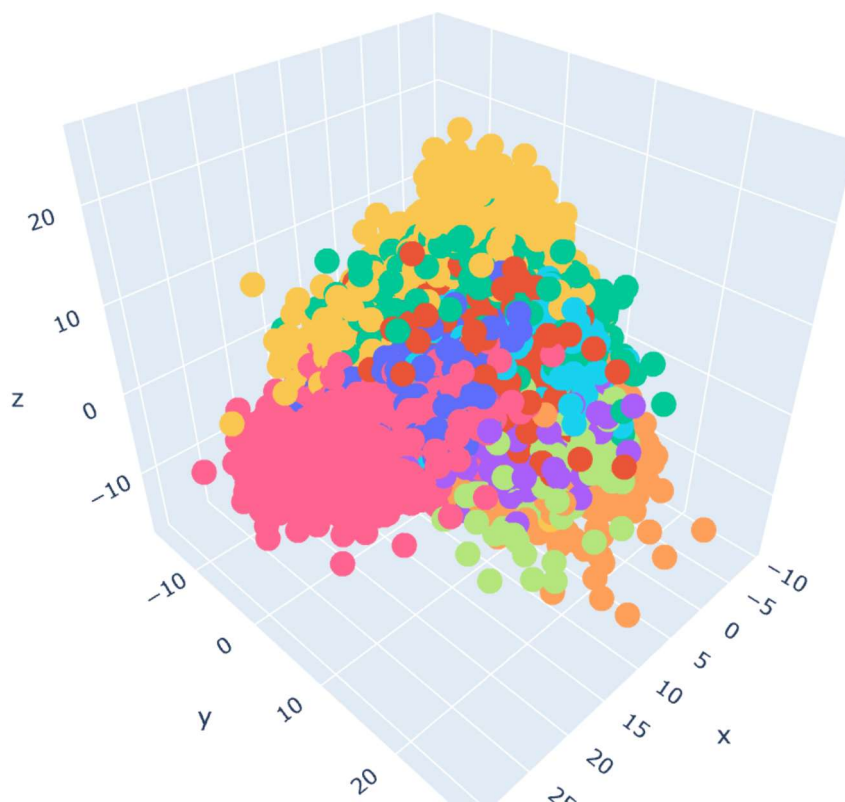
```
X_train_trf
```

```
array([[ -2.71863253,  -0.48982409,   1.1354966 ],
       [-0.67698015,  -6.75362491,  -2.33590139],
       [-3.03323281,   6.50981624,   7.49182983],
       ...,
       [ 2.14883643,   0.78079444,  -0.74738774],
       [ 1.05957006,   0.94767312,   3.94973114],
       [17.70258446,   1.96188185,  -4.94352837]])
```

```
X_train_trf.shape
```

```
(33600, 3)
```

```
y_train_trf = y_train.astype(str)
fig = px.scatter_3d(df1,x=X_train_trf[:,0],y= X_train_trf[:,1],z=X_train_trf[:,2],color = y_train_trf)
fig.update_layout(margin=dict(l=20, r=20, t=20, b=20))
fig.show()
```



```
pca1.explained_variance_
# eigen value
```

```
array([40.87278367, 28.96628019, 27.04280036])
```

```
pca1.components_.shape
# eigen vector
```

```
(3, 784)
```

```
pca1.explained_variance_ratio_
```

```
array([0.06054506, 0.0429079 , 0.04005864])
```

```
pca1.explained_variance_ratio_.shape
```

```
(3,)
```

```
pca = PCA(n_components=None)
X_train_trf3 = pca.fit_transform(X_train)
X_test_trf3 = pca.fit_transform(X_test)
```

```
pca.explained_variance_
```

[illegible]

pca.explained_variance_.shape

(784,)

pca.components

```
array([[ 1.00000000e+00, -2.71689802e-16, -2.63138480e-16, ...,
         2.11049931e-17,  3.84350720e-17, -5.23637170e-17],
       [ 2.71689802e-16,  1.00000000e+00, -9.15933995e-15, ...,
        6.85802947e-17,  3.30688889e-17, -1.30972595e-17],
       [ 2.63138480e-16,  9.46465128e-15,  1.00000000e+00, ...,
        1.77467598e-16,  3.93541502e-17, -8.03850830e-17],
       ...,
```



```
[-0.00000000e+00,  7.75544526e-17,  6.84035269e-17, ...,  
 1.58695808e-01,  3.34354779e-01, -6.54136393e-02],  
[-1.29449650e-16,  1.11022302e-16, -1.38777878e-17, ...,  
 1.76581879e-01, -1.36368123e-01, -4.59598879e-01],  
[-3.67604351e-16,  4.44089210e-16, -4.99600361e-16, ...,  
 -4.76999076e-03,  1.52576752e-01,  1.93991089e-02]])
```

pca.components_.shape

(784, 784)

pca.explained_variance_ratio_

```
0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,  
0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,  
0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,  
0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,  
0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00])
```

```
pca.explained_variance_ratio_.shape
```

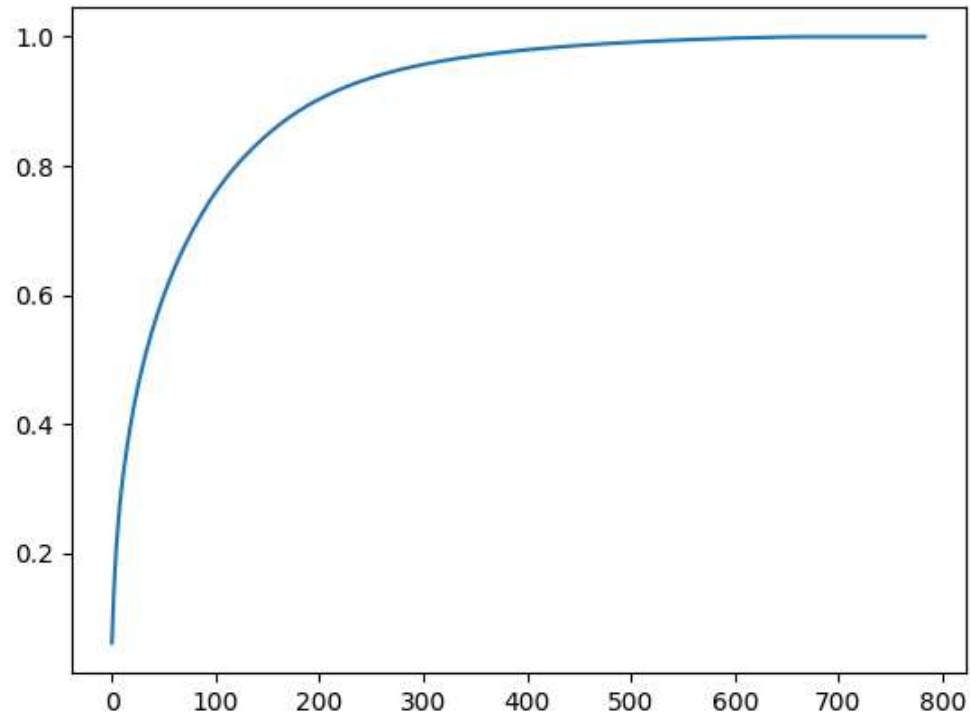
```
(784,)
```

```
np.cumsum(pca.explained_variance_ratio_)
```

```
1.      , 1.      , 1.      , 1.      , 1.      ,  
1.      , 1.      , 1.      , 1.      , 1.      ,  
1.      , 1.      , 1.      , 1.      , 1.      ,  
1.      , 1.      , 1.      , 1.      , 1.)
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
```

```
[<matplotlib.lines.Line2D at 0x7e2bb87e7ef0>]
```



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.